A2 Design Document

1. Introduction

This document contains a brief overview of our planned implementation of the second assignment. It should not be considered a perfected blueprint of any kind, but rather a rough draft, since it is hard to say in detail how every class is going to look by the end.

Much of the overall structure was at least in part inspired by the structure of assignment 1. This is because many parts of it function in much the same way. For example, the CommandLine class will likely resemble our own CommandLine class, and trying to do anything all to different would be, in many ways, unnecessary.

What follows will be a brief description of classes we are confident will be in our implementation. Keep in mind that every class will obviously possess necessary getters and setters even if they are omitted for the sake of clarity. We also omit detailed descriptions of say, specific abilities, and instead just say “X piece has a unique ability”. Furthermore, there is a possible distinction of ActivePiece and PassivePiece that could be made.

In an attempt to avoid really verbose nesting of classes (it would take like 5 sub-headings to go all the way down to the individual piece types) the inheritance chain will be kept simple to read.

2. Classes

2.1 Game

The Game class is an instance of the game itself. It handles the macro logic of the game, and ensure it starts and terminates correctly. It must validate and load config files and also provide important data related to the proper execution of the game logic, like providing the currently active player and the next player. It will contain a CommandLine instance, a Board instance, and the Player instances.

2.2 Entity

Entity is the parent class of Player and Piece. It will likely end up as an abstract class and possess methods that are mostly overriden. Aside from an ID it might not end up possessing many members.

2.2.1 Player

The Player class contains all the data related to each of the two players. It will contain an ID as well as relevant data, like the mana.

2.2.2 Piece

Piece is a subclass of Entity. Each Piece has every type of chess piece as a further subclass, and those individual pieces have further special types. The full chain therefore looks like this: Entity => Piece => PieceType (e.g. Pawn) => SpecialPieceType (e.g. GoldenPawn). This structure holds for all pieces. As mentioned in the introduction, there is a further distinction to be made between active pieces and passive ones, but we are as of yet unsure if we should make an entirely separate class for this. There would obviously be a method to play or move the piece. This will likely be implemented as a virtual method which is implemented by the SpecialPieceType.

2.3 Board

The Board will be a field of slots populated by the chess pieces. It will possess a list of squares, as well as a bool that indicates whether it is currently printed or not. Methods will include a print menthod, a togglePrint method and methods related to special squares.

2.3.1 Square

A Square on the game board.

2.4 Command

Command will be similar to A1. It will contain an enum class CommandType and methods related to config files. Members will include a CommandType and the list of parameters.

2.5 CommandLine

CommandLine will likely be the class most similar to A1, since it contains essentially everything that we could need. It will parse and format the command line inputs and make them readily usable.

2.6 Item

Item is the class related to the usable components of the game. It will possess an ID, a displayname string, and a virtual use that is overridden by the subclasses Potion and Tool.

2.6.1 Potion

Subclass of Item related to consumable potions. Will override use.

2.6.2 Tool

Subclass of Item related to usable tools. Will override use.

2.6 Coordinates

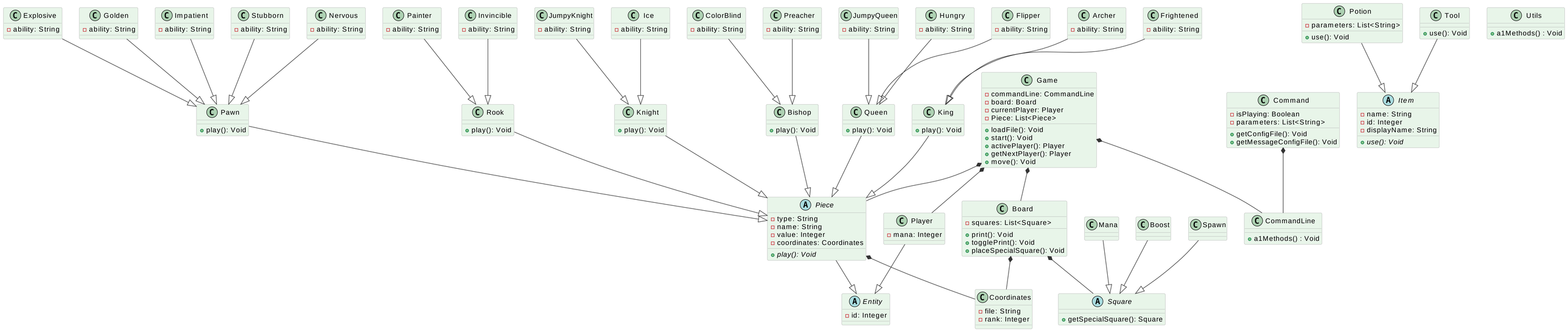
A coordinate class reminiscent of A1. Will contain x and y values, likely as file and rank.

2.7 Utils

Utils class once again similar to A1. Will contain miscellaneous functionality

3. Diagram

The next page will contain the UML diagram.


Explosive
- ability: String
Golden
- ability: String
Impatient
- ability: String
Stubborn
- ability: String
Nervous
- ability: String
Painter
- ability: String
Invincible
- ability: String
JumpyKnight
- ability: String
Ice
- ability: String
ColorBlind
- ability: String
Preacher
- ability: String
JumpyQueen
- ability: String
Hungry
- ability: String
Flipper
- ability: String
Archer
- ability: String
Frightened
- ability: String
Potion
- parameters: List<String>
+ use(): Void
Tool
+ use(): Void
Utils
+ a1Methods() : Void
Pawn
+ play(): Void
Rook
+ play(): Void
Knight
+ play(): Void
Bishop
+ play(): Void
Queen
+ play(): Void
King
+ play(): Void
Game
- commandLine: CommandLine
- board: Board
- currentPlayer: Player
- Piece: List<Piece>
+ loadFile(): Void
+ start(): Void
+ activePlayer(): Player
+ getNextPlayer(): Player
+ move(): Void
Command
- isPlaying: Boolean
- parameters: List<String>
+ getConfigFile(): Void
+ getMessageConfigFile(): Void
Item
- name: String
- id: Integer
- displayName: String
+ use(): Void
Piece
- type: String
- name: String
- value: Integer
- coordinates: Coordinates
+ play(): Void
Player
- mana: Integer
Board
- squares: List<Square>
+ print(): Void
+ togglePrint(): Void
+ placeSpecialSquare(): Void
Mana
Boost
Spawn
CommandLine
+ a1Methods() : Void
Entity
- id: Integer
Coordinates
- file: String
- rank: Integer
Square
+ getSpecialSquare(): Square